Evidencia de todos los tests de la carpeta User (23 tests)

Karate Labs

7

0

dev

Features

2026-02-25 11:30:06 a. m.

Tags | Timeline

Feature	Title	Passed	Failed	Scenarios	Time (ms)
petstore/users/CreateListOfUsers.feature	Crear usuarios usando lista	4	0	4	2203
petstore/users/CreateUser.feature	Crear usuario individual	2	0	2	848
petstore/users/DeleteUser.feature	Eliminar un usuario por el username	4	0	4	1988
petstore/users/GetUserByUsername.feature	Buscar usuario por username	4	0	4	2105
petstore/users/Login.feature	Login (Aqui lanza 200 para cualquier username y password, asi que no e sposible hacer validaciones)	3	0	3	1162
petstore/users/logout.feature	Cerrar sesión	1	0	1	1046
petstore/users/UpdateUser.feature	Actualizar usuario	5	0	5	10816

Karate Labs

4

0

dev

Scenarios

2026-02-25 11:29:47 a. m.

[1:10] Crear múltiples usuarios válidos

[2:19] Crear solo un usuario

[3:29] Crear usuarios enviando lista vacía

[4:39] Enviar estructura incorrecta

Summary | Tags | Feature: *petstore/users/CreateListOfUsers.feature* | **Crear usuarios usando lista**

User ValidUsers HappyPath CreateWithList — ms: 1165

Scenario: [1:10] **Crear múltiples usuarios válidos**

>>	Background:	
12	* def users = JsonCrearUsuarios	1
13	Given path 'user', 'createWithList'	7
14	And request users	0
15	When method post	1124
16	Then status 200	0

User HappyPath CreateWithList ValidOneUsers — ms: 336

Scenario: [2:19] **Crear solo un usuario**

>>	Background:	
21	* def users = JsonCrearUnUsuario	0
22	Given path 'user', 'createWithList'	0
23	And request users	0
24	When method post	333
25	Then status 200	0

EmptyList User HappyPath CreateWithList — ms: 372

Scenario: [3:29] **Crear usuarios enviando lista vacía**

>>	Background:	
31	* def users = []	0
32	Given path 'user', 'createWithList'	0
33	And request users	0

Karate Labs

2

0

dev

Scenarios

2026-02-25 11:29:48 a. m.

[1:10] Crear usuario válido correctamente

[2:22] Crear usuario enviando body vacío

Summary | Tags | Feature: *petstore/users/CreateUser.feature* | **Crear usuario individual**

ValidUser User CreateUser HappyPath — ms: 475

Scenario: [1:10] **Crear usuario válido correctamente**

>>	Background:	
12	* def user = JsonCrearUsuario	0
14	Given path 'user'	0
15	And request user	0
16	When method post	474
17	Then status 200	0

User CreateUser EmptyBody UnHappyPath — ms: 372

Scenario: [2:22] **Crear usuario enviando body vacío**

>>	Background:	
24	Given path 'user'	0
25	And request {}	0
26	When method post	371
27	Then status 200	0

Karate Labs

4

0

dev

Scenarios

2026-02-25 11:29:51 a. m.

- [1:10] Eliminar usuario existente correctamente
- [2:28] Eliminar usuario que no existe
- [3:38] Eliminar usuario con username con solo un espacio
- [4:46] Eliminar usuario con caracteres especiales

Summary | Tags | Feature: *petstore/users/DeleteUser.feature* | **Eliminar un usuario por el username**

User DeleteExistingUser DeleteUser HappyPath — ms: 870

Scenario: [1:10] **Eliminar usuario existente correctamente**

>>	Background:	
12	* def user = JsonUsuario	1
13	* def username = user.username	1
	# Crear usuario	
16	Given path 'user'	0
17	And request user	0
18	When method post	422
19	Then status 200	0
	# Eliminar usuario	
22	Given path 'user', username	5
23	When method delete	439
24	Then status 200	0

User DeleteUser UnHappyPath DeleteNonExistingUser — ms: 438

Scenario: [2:28] **Eliminar usuario que no existe**

>>	Background:	
30	* def nonExistingUsername = 'userDoesNotExistXYZ123XYZ'	1
32	Given path 'user', nonExistingUsername	1
33	When method delete	435
34	Then status 404	0

Karate Labs

4

0

dev

Scenarios

2026-02-25 11:29:53 a. m.

- [1:9] Buscar usuario existente por username
- [2:30] Buscar usuario que no existe
- [3:38] Buscar usuario con username con solo un espacio
- [4:47] Buscar usuario con caracteres especiales

Summary | Tags | Feature: *petstore/users/GetUserByUsername.feature* | **Buscar usuario por username**

GetUserByUsername User ExistingUser HappyPath — ms: 1030

Scenario: [1:9] **Buscar usuario existente por username**

>>	Background:	
11	* def users = JsonCrearUnUsuario	0
12	* def username = users[0].username	1
13	* def email = users[0].email	2
	# Crear usuario usando lista	
16	Given path 'user', 'createWithList'	0
17	And request users	0
18	When method post	680
19	Then status 200	0
	# Buscar usuario creado	
22	Given path 'user', username	2
23	When method get	327
24	Then status 200	0
25	And match response.username == username	16
26	And match response.email == email	1

GetUserByUsername NonExistingUser User UnHappyPath — ms: 326

Scenario: [2:30] **Buscar usuario que no existe**

>>	Background:	
32	Given path 'user', 'usuarioInexistenteXYZ'	0
33	When method get	322

Karate Labs

5

0

dev

Scenarios

2026-02-25 11:30:06 a. m.

- [1:9] Actualizar usuario existente correctamente
- [2:32] Actualizar usuario que no existe
- [3:44] Username en path diferente al del body
- [4:64] Enviar estructura inválida en update
- [5:80] Actualizar usuario con username con solo un espacio

Summary | Tags | Feature: *petstore/users/UpdateUser.feature* | **Actualizar usuario**

User UpdateUser HappyPath UpdateExistingUser — ms: 727

Scenario: [1:9] **Actualizar usuario existente correctamente**

>>	Background:	
11	* def user = JsonUsuario	0
12	* def username = user.username	0
	# Crear usuario primero	
15	Given path 'user'	0
16	And request user	0
17	When method post	358
18	Then status 200	0
	# Modificamos datos	
21	* set user.firstName = "NombreActualizado"	3
22	* set user.email = "nuevo@email.com"	1
	# Hacemos update	
25	Given path 'user', username	1
26	And request user	0
27	When method put	362
28	Then status 200	0

User UpdateUser UnHappyPath UpdateNonExistingUser — ms: 7396

Scenario: [2:32] **Actualizar usuario que no existe**

>>	Background:	
34	* def user = JsonUsuario	0
35	* set user.username = "userDoesNotExistXYZ"	1

Karate Labs

3

0

dev

Scenarios

2026-02-25 11:29:54 a. m.

[1:10] Login con credenciales válidas

[2:22] Login con usuario que no existe

[3:31] Login con username vacío

Summary | Tags | Feature: ***petstore/users/Login.feature*** | **Login (Aqui lanza 200 para cualquier username y password, asi que no e sposible hacer validaciones)**

User ValidLogin Login HappyPath

Scenario: [1:10] **Login con credenciales válidas** — ms: 367

>>	Background:	
12	Given path 'user', 'login'	0
13	And param username = "username"	1
14	And param password = "password"	0
15	When method get	363
16	Then status 200	0
17	And match response.message contains 'logged'	1

LoginNonExistingUser User UnHappyPath Login

Scenario: [2:22] **Login con usuario que no existe** — ms: 352

>>	Background:	
24	Given path 'user', 'login'	0
25	And param username = 'userDoesNotExistXYZ'	1
26	And param password = '12345'	1
27	When method get	348
28	Then status 200	0

User EmptyUsername UnHappyPath Login

Scenario: [3:31] **Login con username vacío** — ms: 443

>>	Background:	